

Advanced Computer Networks Lab



Dr Sudipta Saha

Associate Professor

Dept of Computer Science & Engineering

Indian Institute of Technology Bhubaneswar

Lab Assignment - 3.3

Teaching Assistant: Arindam Pal



**DSSRG: Decentralized
Smart Systems Research
Group**

<https://sites.google.com/iitbbs.ac.in/dssrg>

Or Google dssrg iitbbs



About

- Transport layer protocols: TCP vs UDP
- TCP is connection oriented, ensures data integrity through ordering, acknowledgement and retransmission.
- UDP is fast, connectionless, and unreliable protocol.
- TCP has everything, why UDP then?
- UDP gives blank slate. Implement additional functionality as required.
- This assignment focuses on such applications that can be created on top of UDP.



Learning outcome (A3.3)

- How a multiplayer game works.
- How to create a simple multiplayer server using UDP.



Assignment 3.3

Problem Statement:

You have to create a multiplayer game server provided a game client. The communication between the server and client would be over UDP. Multiple client should get connected to the server and clients should be able to interact with each other in-game.

A windows client.exe is provided as well as linux executable in client.zip



Assignment 3.3 (packet from client to server)

Client to server

CONNECT

uint8_t packet_type [1] (1 byte)	float x (8 byte)	float y (8 byte)
-------------------------------------	---------------------	---------------------

PLAYER_ID_ACK

uint8_t packet_type [2] (1 byte)	float x (8 byte)	float y (8 byte)
-------------------------------------	---------------------	---------------------

CONN_CLOSE

uint8_t packet_type [3] (1 byte)	uint8_t player_id (1 byte)
-------------------------------------	-------------------------------

PLAYER_INPUT

uint8_t packet_type [4] (1 byte)	uint8_t player_id (1 byte)	float x (8 byte)	float y (8 byte)	uint8_t dir (1 byte)	int8_t jumped (1 byte)
-------------------------------------	-------------------------------	---------------------	---------------------	-------------------------	---------------------------

PLAYER_POS

uint8_t packet_type [5] (1 byte)	uint8_t player_id (1 byte)	float x (8 byte)	float y (8 byte)
-------------------------------------	-------------------------------	---------------------	---------------------

Assignment 3.3 (packet from server to client)

Server to client

PLAYER_ID

uint8_t packet_type [1] (1 byte)	uint8_t player_id (1 byte)
-------------------------------------	-------------------------------

SPAWN_PLAYER

uint8_t packet_type [2] (1 byte)	uint8_t player_id (1 byte)	float x (8 byte)	float y (8 byte)
-------------------------------------	-------------------------------	---------------------	---------------------

DESPAWN_PLAYER

uint8_t packet_type [3] (1 byte)	uint8_t player_id (1 byte)
-------------------------------------	-------------------------------

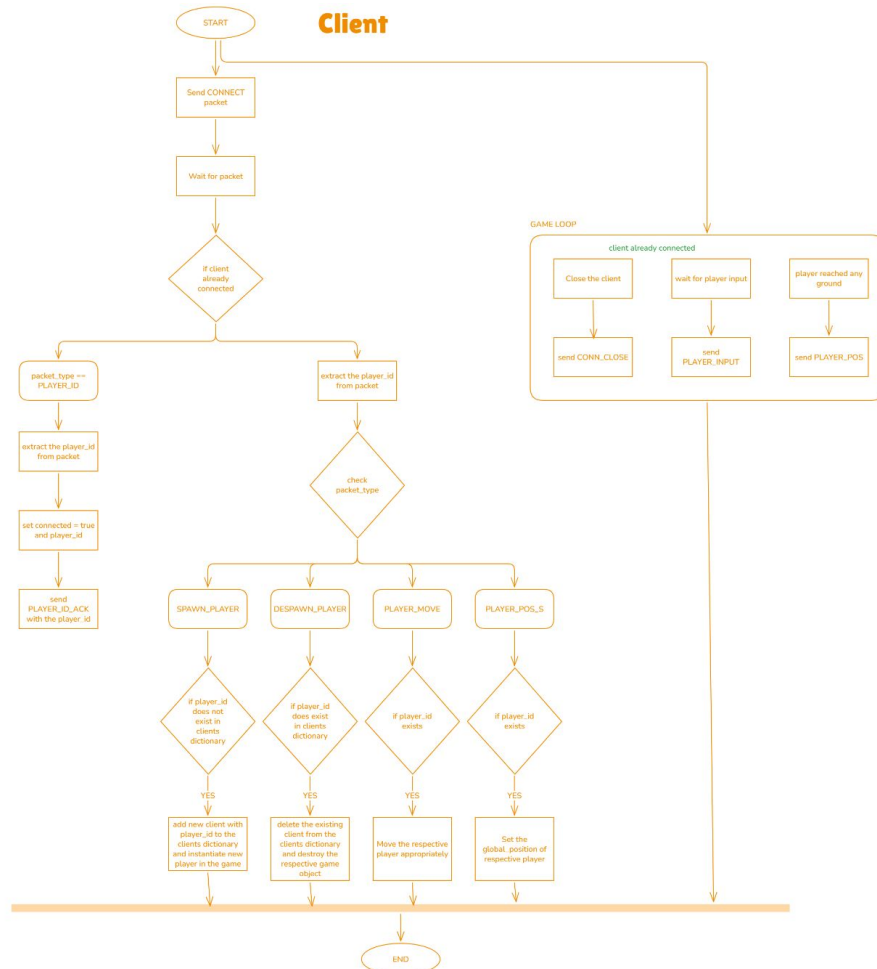
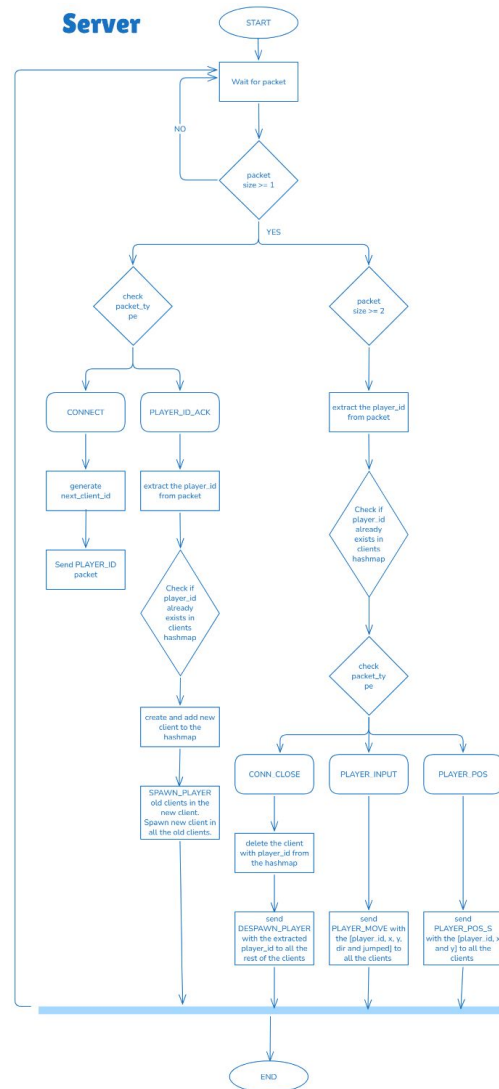
PLAYER_MOVE

uint8_t packet_type [4] (1 byte)	uint8_t player_id (1 byte)	float x (8 byte)	float y (8 byte)	uint8_t dir (1 byte)	int8_t jumped (1 byte)
-------------------------------------	-------------------------------	---------------------	---------------------	-------------------------	---------------------------

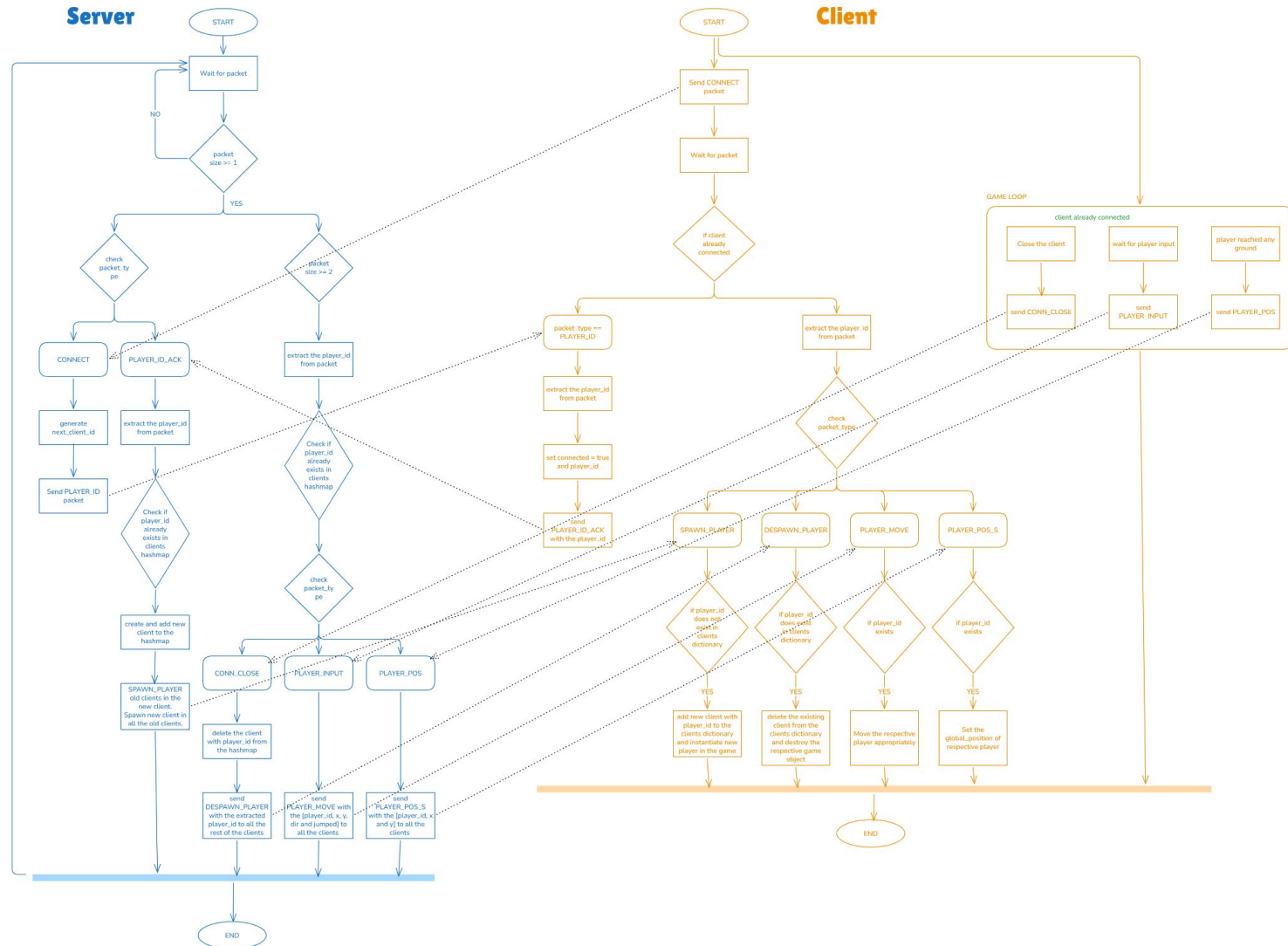
PLAYER_POS_S

uint8_t packet_type [5] (1 byte)	uint8_t player_id (1 byte)	float x (8 byte)	float y (8 byte)
-------------------------------------	-------------------------------	---------------------	---------------------

Assignment 3.3 (architecture)



Assignment 3.3 (architecture with packet traces)



Assignment 3.3 (server flow)

Flow of the server:

- Start the server and wait for packets to come in a infinite while loop.
- When you receive the packet, check the first byte to get which packet_type was received.
- If packet_type == CONNECT
 - Generate the next_player_id
 - Send PLAYER_ID packet to the client who sent the CONNECT packet
- If packet_type == PLAYER_ID_ACK
 - This means that you are getting ACK for the PLAYER_ID packet you sent previously
 - Now you create a new client in the hashmap named clients, with key as player_id you receive from PLAYER_ID_ACK packet
 - Add the new client if the player_id does not already exists
 - Send SPAWN_PLAYER packets to all the clients
 - to new client: SPAWN_PLAYER of all old clients
 - to all old clients: SPAWN_PLAYER of the new client



Assignment 3.3 (server flow)

- If packet size ≥ 2 then the packet is from already connected client
 - Extract player_id from the second byte of the packet
 - Check if player_id exists in the clients hashmap, if not then continue to the next loop iteration.
 - Check packet_type
 - If packet_type == CONN_CLOSE
 - Delete the client of player_id from the clients hashmap
 - Send DESPAWN_PLAYER with player_id to the rest of the connected clients
 - If packet_type == PLAYER_INPUT
 - Send PLAYER_MOVE with [player_id, x, y, dir, and jumped] to all the connected clients
 - If packet_type == PLAYER_POS
 - Send PLAYER_POS_S with [player_id, x, and y] to all the connected clients



Assignment 3.3 (hints)

- Create enums to store SendMessage and RecvMessage packet_types
- Create a hashmap using `std::unordered_map` for storing clients.
 - `std::unordered_map<uint8_t, struct Client> clients;`
- Create a struct for client to store details regarding each client like its id, `current_position.x`, `current_position.y`, address

```
struct Client
{
    uint8_t id;
    double x;
    double y;
    struct sockaddr_storage addr;
};
```

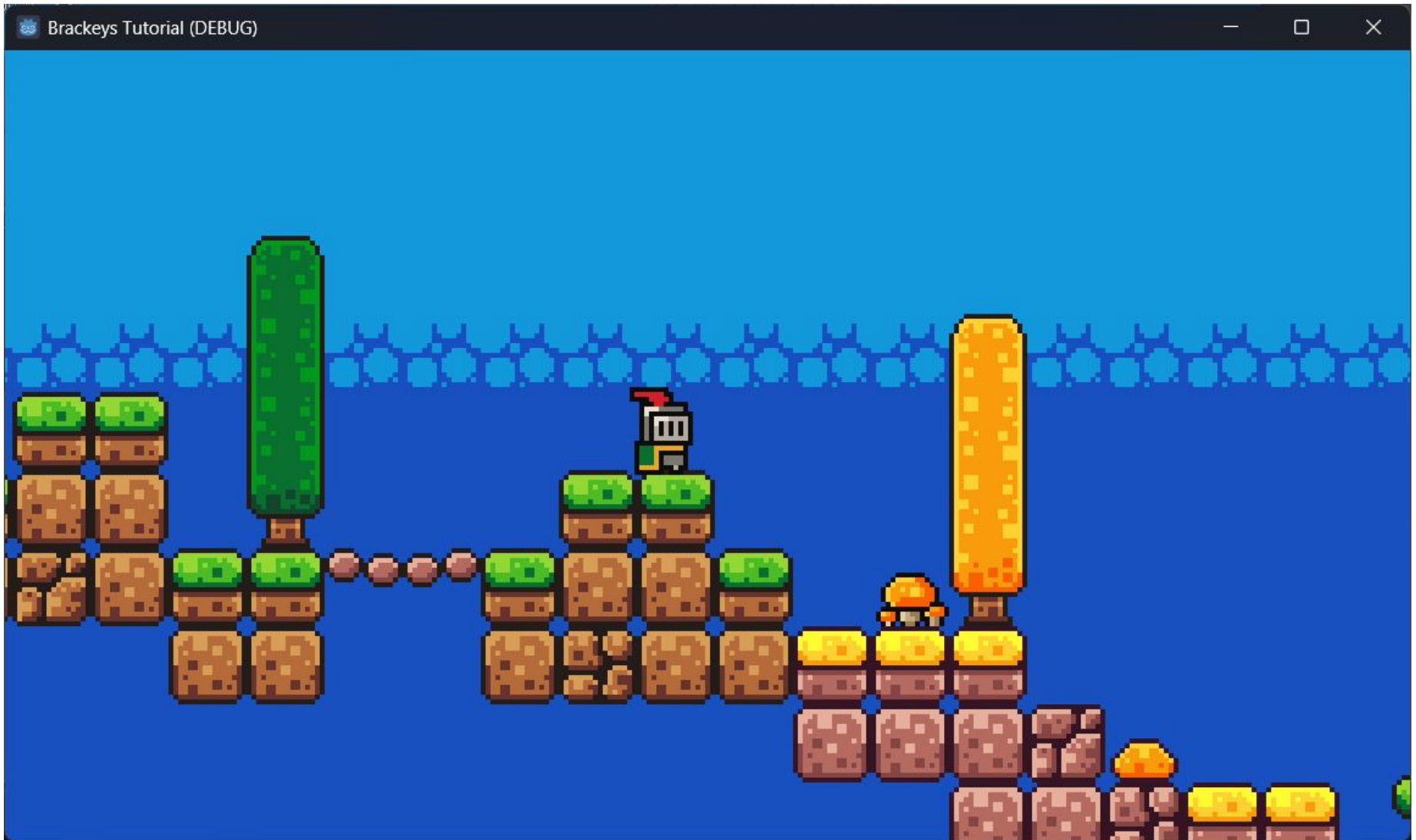


Assignment 3.3 (hints)

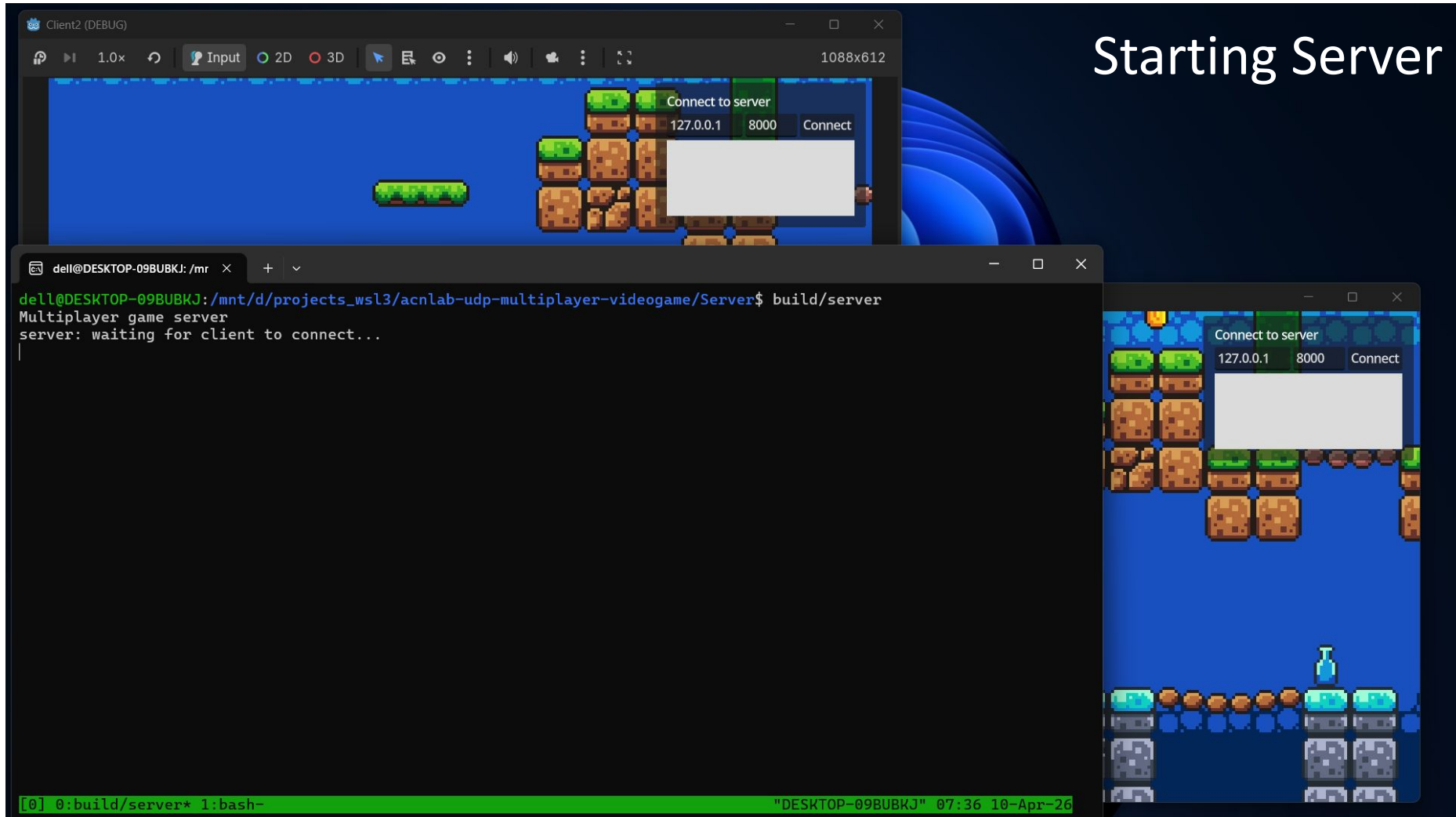
- The video game client is exported for both Windows and Linux operating systems. So you can run the client on both operating systems.
- But the server code you'll be writing following unix socket programming, so you can build and run only on linux machines.
- The windows executable is the client.exe provided to you. Its a standalone binary.
- Linux binary is present in the client.zip archive, extract it and you'll get the binary.
- For both executables, you can directly run them by double clicking them in their appropriate operating system.



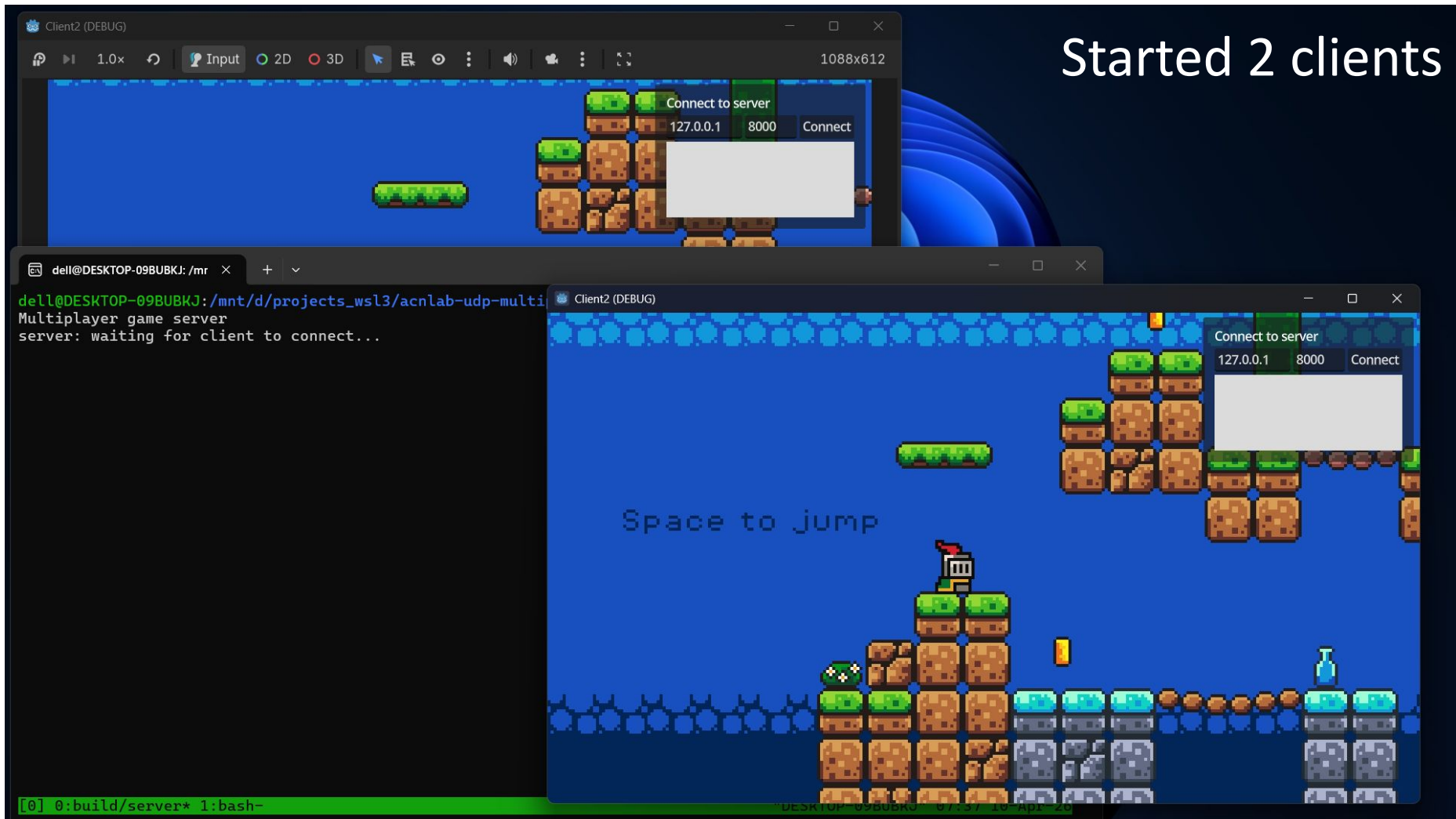
Assignment 3.3 (demo)



Assignment 3.3 (demo)



Assignment 3.3 (demo)



Assignment 3.3 (demo)

Client 1 connected

The screenshot displays a multiplatform environment. On the left, a terminal window titled 'dell@DESKTOP-09BUBKJ: /mr' shows the server's log. The log indicates that the server is waiting for a client, receives a connection message, sends a player ID, and successfully connects a new client. The terminal output is as follows:

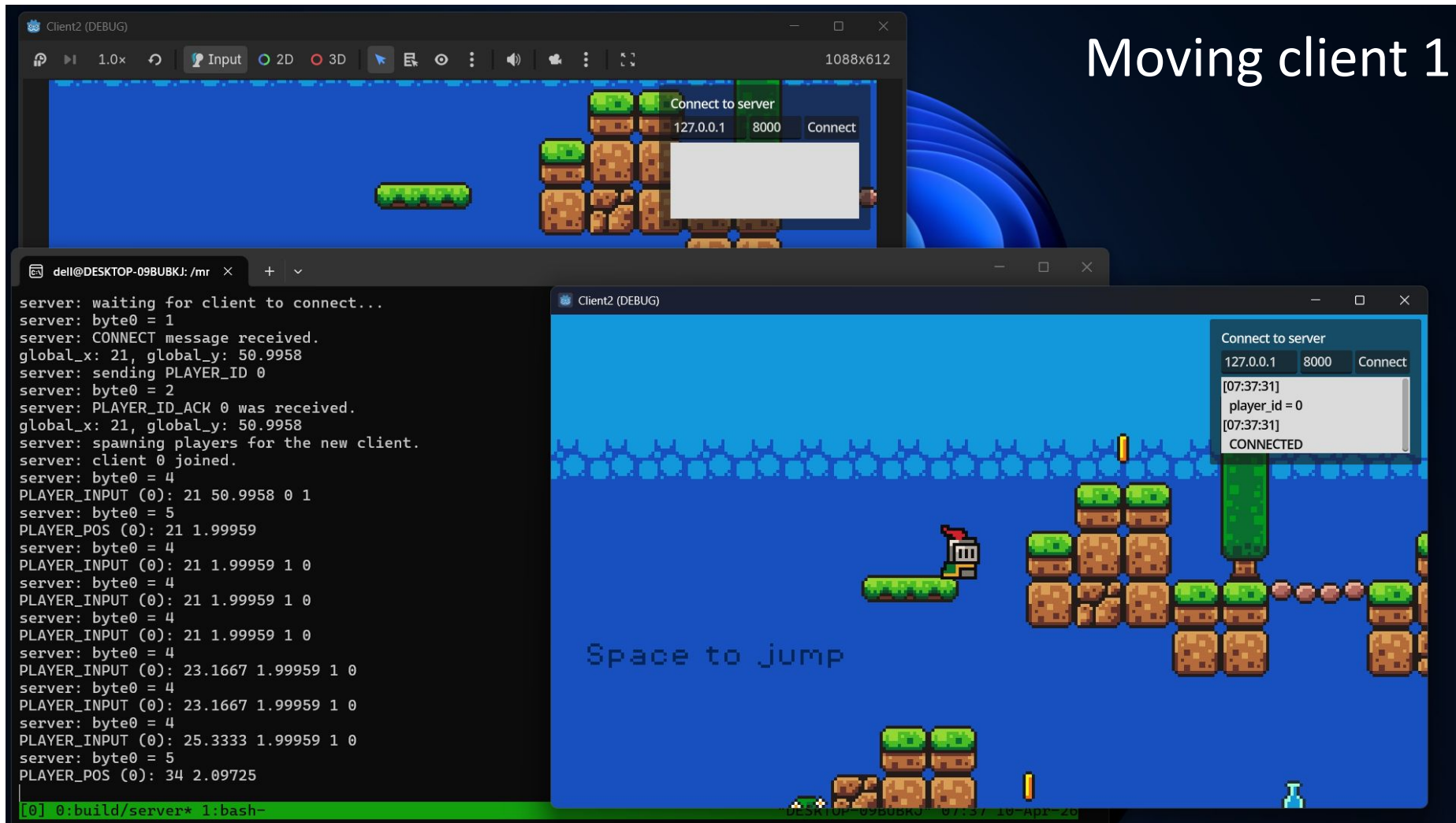
```
dell@DESKTOP-09BUBKJ: /mnt/d/projects_wsl3/acnlab-udp-multi
Multiplayer game server
server: waiting for client to connect...
server: byte0 = 1
server: CONNECT message received.
global_x: 21, global_y: 50.9958
server: sending PLAYER_ID 0
server: byte0 = 2
server: PLAYER_ID_ACK 0 was received.
global_x: 21, global_y: 50.9958
server: spawning players for the new client.
server: client 0 joined.
```

On the right, two instances of a Super Mario Bros.-style game are shown. The top window, titled 'Client2 (DEBUG)', shows a 'Connect to server' dialog with the IP '127.0.0.1' and port '8000'. The bottom window, also titled 'Client2 (DEBUG)', shows the same dialog with the 'Connect' button pressed, and a log indicating 'player_id = 0' and 'CONNECTED'. The game level features a blue sky, brown brick platforms, green pipes, and a Goomba enemy. The text 'Space to jump' is visible on the screen.



Assignment 3.3 (demo)

Moving client 1



The screenshot displays a game client window titled 'Client2 (DEBUG)' and a terminal window showing server logs. The game client window shows a Super Mario Bros. style game with a 'Connect to server' dialog box. The terminal window shows the server's log of events, including the connection of a new client.

Client2 (DEBUG) Dialog Box:

Connect to server		
127.0.0.1	8000	Connect

Terminal Output:

```
server: waiting for client to connect...
server: byte0 = 1
server: CONNECT message received.
global_x: 21, global_y: 50.9958
server: sending PLAYER_ID 0
server: byte0 = 2
server: PLAYER_ID_ACK 0 was received.
global_x: 21, global_y: 50.9958
server: spawning players for the new client.
server: client 0 joined.
server: byte0 = 4
PLAYER_INPUT (0): 21 50.9958 0 1
server: byte0 = 5
PLAYER_POS (0): 21 1.99959
server: byte0 = 4
PLAYER_INPUT (0): 21 1.99959 1 0
server: byte0 = 4
PLAYER_INPUT (0): 21 1.99959 1 0
server: byte0 = 4
PLAYER_INPUT (0): 21 1.99959 1 0
server: byte0 = 4
PLAYER_INPUT (0): 23.1667 1.99959 1 0
server: byte0 = 4
PLAYER_INPUT (0): 23.1667 1.99959 1 0
server: byte0 = 4
PLAYER_INPUT (0): 25.3333 1.99959 1 0
server: byte0 = 5
PLAYER_POS (0): 34 2.09725
[0] 0:build/server* 1:bash-
```



Assignment 3.3 (demo)

Connected client 2

```
Client2 (DEBUG)
1088x612
Connect to server
127.0.0.1 8000 Connect
[07:38:03]
player_id = 1
[07:38:03]
CONNECTED

Space to jump

PLAYER_POS (0): 34 2.09725
server: byte0 = 1
server: CONNECT message received.
global_x: 21, global_y: 50.9958
server: sending PLAYER_ID 1
server: byte0 = 2
server: PLAYER_ID_ACK 1 was received.
global_x: 21, global_y: 50.9958
server: spawning players for the new client.
server: sending SPAWN_PLAYER [1 21 50.9958] to client 0
server: client 0 position = (34, 2.09725)
server: sending SPAWN_PLAYER [0 34 2.09725] to client 1
server: client 1 joined.
[0] 0:build/server* 1: bash-
```

```
Client2 (DEBUG)
1088x612
Connect to server
127.0.0.1 8000 Connect
[07:37:31]
player_id = 0
[07:37:31]
CONNECTED

Space to jump

PLAYER_POS (0): 34 2.09725
server: byte0 = 1
server: CONNECT message received.
global_x: 21, global_y: 50.9958
server: sending PLAYER_ID 1
server: byte0 = 2
server: PLAYER_ID_ACK 1 was received.
global_x: 21, global_y: 50.9958
server: spawning players for the new client.
server: sending SPAWN_PLAYER [1 21 50.9958] to client 0
server: client 0 position = (34, 2.09725)
server: sending SPAWN_PLAYER [0 34 2.09725] to client 1
server: client 1 joined.
[0] 0:build/server* 1: bash-
```

Assignment 3.3 (demo)

Client2 (DEBUG)

1088x612

Input 2D 3D

Connect to server
127.0.0.1 8000 Connect
[07:38:03] player_id = 1
[07:38:03] CONNECTED

jump

PLAYER_INPUT (1): 55.6907 -46.4058 1 0
server: byte0 = 4
PLAYER_INPUT (1): 57.8573 -46.5058 1 0
server: byte0 = 4
PLAYER_INPUT (1): 60.024 -46.3336 1 0
server: byte0 = 4
PLAYER_INPUT (1): 60.024 -45.8891 1 0
server: byte0 = 4
PLAYER_INPUT (1): 62.1907 -45.1725 1 0
server: byte0 = 4
PLAYER_INPUT (1): 62.1907 -44.1836 1 0
server: byte0 = 5
PLAYER_POS (1): 73.024 -29.0145

[0] 0:build/server* 1:bash-

Moving client 2

Connect to server
127.0.0.1 8000 Connect
[07:37:31] player_id = 0
[07:37:31] CONNECTED

Space to jump

DESKTOP-09B0BKO 07:38 10-Apr-20



Thank you

